Laporan Tugas Besar II IF 3058 Kriptografi

Implementasi Program Tanda Tangan Digital dengan Menggunakan Algoritma RSA dan Fungsi *Hash* SHA-1

"TEEKEN"



Dipersiapkan oleh:

Jonathan Marcel (13507072)

Samsu Sempena (13507088)

Aris Feryanto (13507110)

Program Studi Teknik Informatika

Institut Teknologi Bandung

Jalan Ganeca 10 Bandung

BAB 1 Deskripsi masalah

Isu-isu di dalam pengiriman pesan antara lain ialah keabsahan pengirim (*user authentication*), keaslian pesan (*message integrity*), dan anti-penyangkalan (*nonrepudiation*). Ketiga masalah tersebut dapat diselesaikan dengan teknik otentikasi pesan (*message authentication*). Salah satu teknik otentikasi ialah menandatangani pesan (*message signature*) dengan tanda tangan digital.

Pada tugas besar dua ini, program Teeken mengimplementasikan algoritma RSA dan SHA-1 untuk memberi tanda tangan digital pada dokumen elektronis. Teeken diimplementasikan sebagai program desktop yang dikembangkan dengan WPF(Windows Presentation Foundation) dan add-in pada email client Microsoft Office Outlook. Program Teeken juga menyediakan *key generator* untuk algoritma RSA yang akan menghasilkan kunci publik dan kunci privat.

Dengan Teeken, pengguna dapat memberikan tanda tangan digital pada dokumen elektronis dan juga memvalidasi keaslian tandatangan tersebut.

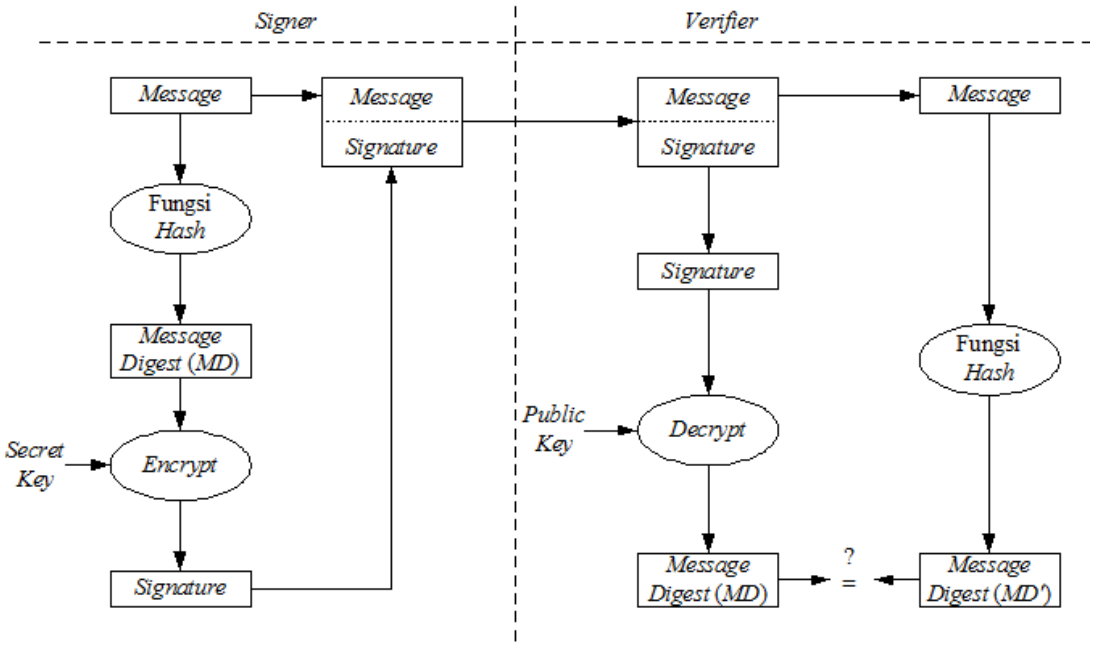
BAB 2 Dasar teori

2.1 Digital Signature

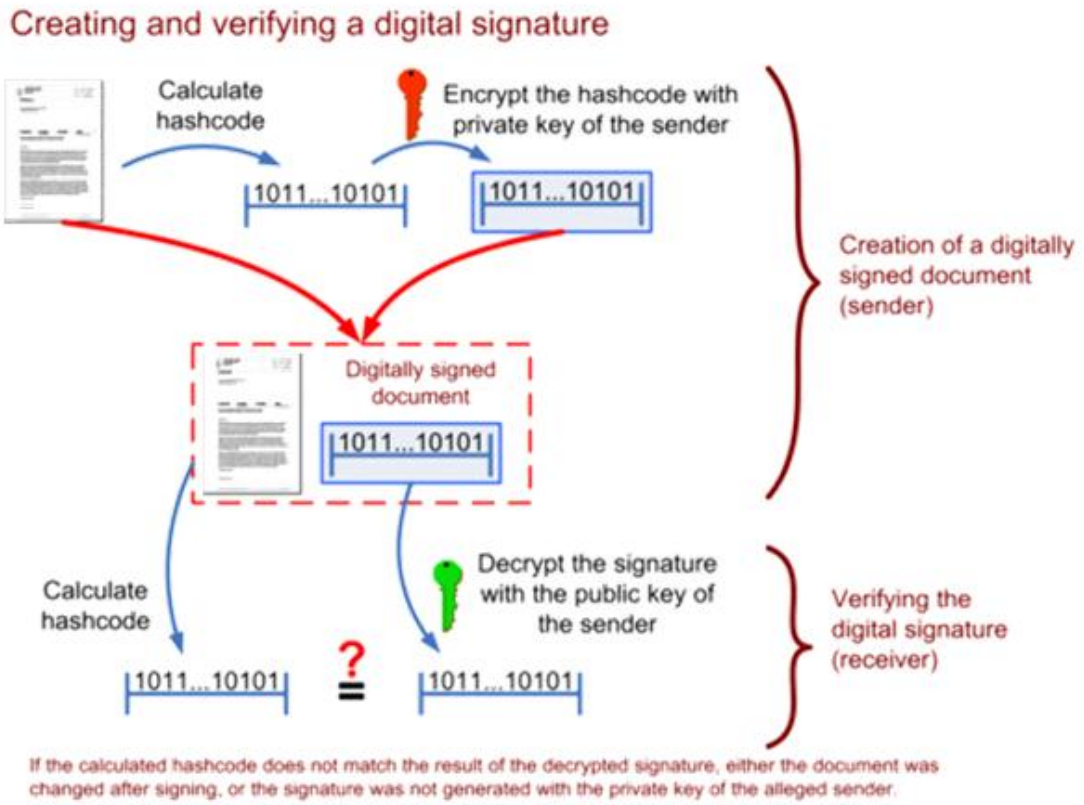
Digital signature (tanda tangan digital) adalah suatu nilai kriptografis yang bergantung pada pesan dan pengirim pesan. Dengan menggunakan tanda tangan digital, maka integritas data dapat dijamin, disamping itu juga digunakan untuk membuktikan keabsahan pengirim dan anti penyangkalan. Proses pemberian tanda tangan digital dapat dilihat pada skema di bawah. Pesan yang hendak dikirim diubah terlebih dahulu menjadi bentuk yang ringkas yang disebut dengan *message digest*. *Message digest* ini diperoleh dengan mentransformasikan pesan dengan menggunakan fungsi hash satu arah. Pesan yang sudah diubah menjadi *message digest* tidak dapat dikembalikan lagi menjadi bentuk semula walaupun digunakan algoritma dan kunci yang sama.

Sembarang pesan yang berukuran apapun diubah oleh fungsi hash menjadi *message digest* yang berukuran tetap. Selanjutnya *message digest* dienkripsikan dengan algoritma kunci publik menggunakan kunci privat pengirim menjadi tanda tangan digital. Di dalam tugas ini digunakan algoritma kunci publik RSA. Kemudian pesan disambung dengan tanda tangan digital, kemudian dikirim melalui saluran komunikasi.

Di tempat penerima, tanda tangan diverifikasi untuk dibuktikan keotentikannya. Mula-mula tanda tangan didekripsi dengan menggunakan kunci publik pengirim pesan, menghasilkan message digest semula. Pengirim kemudian mengubah pesan menjadi *message digest* baru menggunakan fungsi hash satu arah yang sama dengan fungsi hash yang digunakan oleh pengirim. Jika hasilnya sama, berarti tanda tangan yang diterima otentik dan berasal dari pengirim yang benar.



Gambar 1 Skema Tanda Tangan Digital



Gambar 2 Skema Tanda Tangan Digital

2.2Algoritma SHA

SHA adalah fungsi satu-arah yang dibuat oleh NIST dan digunakan bersama DSS (*Digital Signature Standard*). SHA didasarkan pada MD4 yang dibuat oleh Ronald L. Rivest dari MIT.

Keamanan SHA terletak pada rancanganan SHA yang membuatnya sedemikian rupa sehingga secara komputasi tidak mungkin menemukan pesan yang berkoresponden dengan *message digest* yang diberikan.

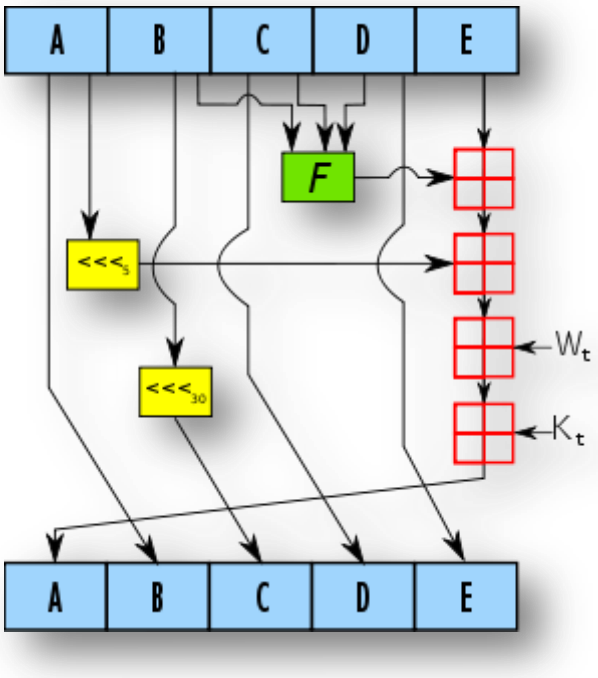
Algoritma SHA menerima masukan berupa pesan dengan ukuran maksimum 2^{64} bit (2.147.483.648 *gugabyte*) dan menghasilkan *message digest* yang panjangnya 160 bit. Lebih panjang dari *message digest* yang dihasilkan MD5.

Langkah-langkah pembuatan *message digest* secara garis besar adalah sebagai berikut:

1. Penambahan bit-bit pengganjal (*padding bits*).
   Pesan ditambah dengan sejumlah bit pengganjal sedemikian sehingga panjang pesan (dalam satuan bit) kongruen dengan 448 modulo 512.
2. Penambahan nilai panjang pesan semula.
   Pesan yang telah diberi bit-bit pengganjal selanjutnya ditambah lagi dengan 64 bit yang menyatakan panjang pesan semula. Setelah ditambah 64 bit, panjang pesan sekarang menjadi 512 bit.

3. Inisialisasi penyangga (*buffer*) MD.
   SHA membutuhkan 5 buah penyangga (*buffer*) yang masing-masing panjangnya 32 bit. Kelima penyangga ini menampung hasil antara dan hasil akhir.
   Nilai kelima penyangga tersebut adalah:
   - A = 67452301
   - B = EFCDAB89
   - C = 98BADCFE
   - D = 10325476
   - E = C3D2E1F0
4. Pengolahan pesan dalam blok berukuran 512 bit.
   Pesan dibagi menjadi L buah blok yang masing-masing panjangnya 512 bit. Setiap blok 512 bit diproses bersama dengan penyangga MD menjadi keluaran 128 bit, dimana proses ini disebut sebagai H_{SHA}. Proses H_{SHA} terdiri dari 80 buah putaran dan masing-masing putaran menggunakan bilangan penambah K_t, yaitu:
   - Putaran $0 \le t \le 19$ K_t = 5A827999
   - Putaran $20 \le t \le 39$ K_t = 6ED9EBA1
   - Putaran $40 \le t \le 59$ K_t = 8F1BBCDC
   - Putaran $60 \le t \le 79$ K_t = CA62C1D6

   Setiap putaran menggunakan operasi dasar yang sama. Operasi dasar tersebut, yaitu:



Gambar 3 Operasi Dasar SHA-1

Operasi dasar yang diperlihatkan pada gambar di atas dapat dituliskan dalam persamaan:

$a, b, c, d, e \leftarrow (CLS_5(a) + f_t(b, c, d) + e, W_t + K_t), a, CLS_{30}(b), c, d$

a, b, c, d, e = lima buah penyangga 32 bit (A, B, C, D, E)

t = putaran, $0 \le t \le 79$

f_t = fungsi logika

CLS_s = *circular shift left* sebanyak *s* bit

W_t = word 32 bit yang diturunkan dari blok 512 bit yang sedang diproses

K_t = konstanta penambah

+ = operasi penjumlahan modulo 2^{32}

Fungsi f_t adalah fungsi logika yang melakukan operasi *bitwise*. Operasi *bitwise* yang dilakukan adalah sebagia berikut:

Putaran	***f_t(b, c, d)***
0 … 19	$(b \wedge c) \vee (\sim b \wedge d)$
20 … 39	b ⊕ c ⊕ d
40 … 59	$(b \wedge c) \vee (b \wedge d) \vee (c \wedge d)$
60 … 79	b ⊕ c ⊕ d

Nilai W_t sampai W_{16} berasal dari 16 *word* pada blok yang sedang diproses. Sedangkan nilai W_t berikutnya didapatkan dari persamaan

$$W_t = W_{t-16} \oplus W_{t-14} \oplus W_{t-8} \oplus W_{t-3}$$

Setelah putaran ke-79, *a, b, c, d,* dan *e* ditambahkan ke *A, B, C, D*, dan *E* dan selanjutnya algoritma memproses data berikutnya. Keluaran akhir dari program SHA adalah hasil penyambungan bit-bit di *A, B, C, D*, dan *E*.

2.3 Algoritma RSA

Algoritma RSA merupakan algoritma kriptografi kunci publik yang paling populer. Algoritma kriptografi kunci publik memiliki dua buah kunci, satu kunci untuk enkripsi (bersifat tidak rahasia atau disebut public key) dan satu kunci untuk dekripsi (bersifat rahasia atau disebut private key).

Algoritma RSA dibuat oleh 3 orang peneliti dari MIT pada tahun 1976, yaitu Ron Rivest, Adi Shamir, dan Leonard Adleman. Sulitnya memfaktorkan bilangan yang besar menjadi faktor-faktor prima merupakan isu yang menyebabkan algoritma RSA memiliki keamanan yang baik. Algoritma RSA menggunakan beberapa besaran antara lain:

1. p dan q bilangan prima
2. n = p*q
3. Φ(n) = (p-1)(q-1)
4. e (kunci enkripsi)
5. d (kunci dekripsi)
6. m (plainteks)
7. c (cipherteks)

Algoritma RSA memiliki dua bagian, yaitu algoritma pembangkitan pasangan kunci dan algoritma enkripsi/dekripsi. Algoritma pembangkitan pasangan kunci ialah sebagai berikut:

1. Pilihlah dua buah bilangan prima sembarang p dan q
2. Hitung n = p*q
3. Hitung Φ(n) = (p-1)(q-1)
4. Pilih kunci publik e, yang relatif prima terhadap Φ(n)
5. Bangkitkan kunci privat d, dengan persamaan

$$d = \frac{1 + k\Phi(n)}{e}$$

Dimana k merupakan bilangan bulat dimulai dari 1, 2, 3, ... sehingga dengan mencoba-coba nilai k diperoleh nilai d yang bulat. Hasil dari algoritma pembangkitan kunci ini ialah e dan d.

Algoritma enkripsi/dekripsi RSA adalah sebagai berikut:

Algoritma enkripsi:

1. Ambil kunci publik penerima pesan e dan modulus n
2. Nyatakan plainteks menjadi blok-blok m_1, m_2, ..., sedemikian sehingga setiap blok merepresentasikan nilai di dalam selang [0, n-1]
3. Setiap blok m_i dienkripsi menjadi blok ci dengan rumus

$$m_i = {m_i}^e mod\ n$$

Algoritma dekripsi:

Setiap blok cipherteks ci dienkripsi kembali menjadi blok m_i dengan rumus

$$m_i = {c_i}^d mod\ n$$

BAB 3 Strategi Penyelesaian Masalah

3.1 Lingkungan Implementasi

Kakas pengembangan yang digunakan antara lain

1. Microsoft Visual Studio Team System 2008
2. Microsoft Visual C# 2008
3. Microsoft Visual Studio Tools For Office 2007

Lingkungan untuk menjalankan program

1. Sistem Operasi Windows XP dan versi selanjutnya
2. Microsoft .NET Framework 3.5 dan versi selanjutnya
3. Microsoft Office Outlook 2007 dan versi selanjutnya

3.2 Trik-Trik Khusus

Pada program add-ins di Office Outlook, secara default Outlook akan memformat isi pesan dengan menambahkan " \r\n" pada bagian pesan yang dianggapnya perlu. Hal ini tentunya akan menyebabkan validasi isi pesan menjadi gagal, untuk mengatasinya pengembang memberikan pengecualian untuk perubahan tersebut sehingga pengecekan dapat tetap berjalan dengan baik.

BAB 4 Struktur Data dan Spesifikasi Subrutin

1. BigInteger

   Struktur data BigInteger yang digunakan adalah kelas BigInteger yang diambil dari http://www.codeproject.com/csharp/biginteger.asp. Struktur data BigInteger ini menggunakan larik (*array*) integer untuk membentuk representasi integer yang lebih besar. Beberapa subrutin yang digunakan dari kelas ini antara lain:

   - `public BigInteger(string value, int radix)`
     Konstruktor bilangan BigInteger dari sebuah string dengan basis `radix`.
   - `public BigInteger genCoPrime(int bits, Random rand)`
     Mengembalikan sebuah bilangan acak dengan jumlah bit yang disediakan sedemikian sehingga FPB antara bilangan yang dikembalikan dengan bilangan ini adalah 1.

2. RSA

   Struktur data RSA digunakan untuk menghasilkan kunci privat dan kunci publik. Kelas ini memanfaatkan kelas BigInteger untuk melakukan perhitungan-perhitungan dalam menghasilkan nilai-nilai berikut:

```
public BigInteger p;
public BigInteger q;
BigInteger n;
BigInteger phi;
BigInteger e;
BigInteger d;
```

   Beberapa subrutin dalam kelas ini, antara lain:

   - `public String inputString(String input)`
     Mengubah string input menjadi string bernilai nomor ASCII dari karakter-karakter string input.
   - `public BigInteger getPrimeNumber(int digit)`
     Mendapatkan bilangan prima yang terdiri dari sejumlah digit.
   - `public void generatePQ(int digit)`
     Menghasilkan nilai p dan q dalam kelas ini.
   - `public void generateKey(out BigInteger privateKey, out BigInteger publicKey,out BigInteger _n)`
   - `public String encrypt(String input, BigInteger _e,BigInteger _n)`
     Mengembalikan signature yang telah dienkripsi dengan kunci privat sebelumnya fungsi generate key telah dipanggil.
   - `public String decrypt(String input, BigInteger _d,BigInteger _n)`
     Menghasilkan hash dengan mendekripsi dari input yang dimasukkan.

3. SHA

   Kelas SHA digunakan untuk menghasilkan nilai hash dari arsip atau pesan yang akan ditandatangan. Fungsi utama yang digunakan untuk menghasilkan hash SHA adalah:

   - `public String HitungHash(List<byte> buffer)`
     Mengembalikan hasil HASH dengan algoritma SHA-1.

4. DigitalSign

   Kelas ini membungkus kelas RSA dan SHA untuk menghasilkan *signature* dan mengecek apakah *signature*-nya benar. Ada dua subrutin utama yang digunakan di kelas ini, yaitu:

   - `public static String sign(byte[] input, BigInteger privateKey, BigInteger n)`

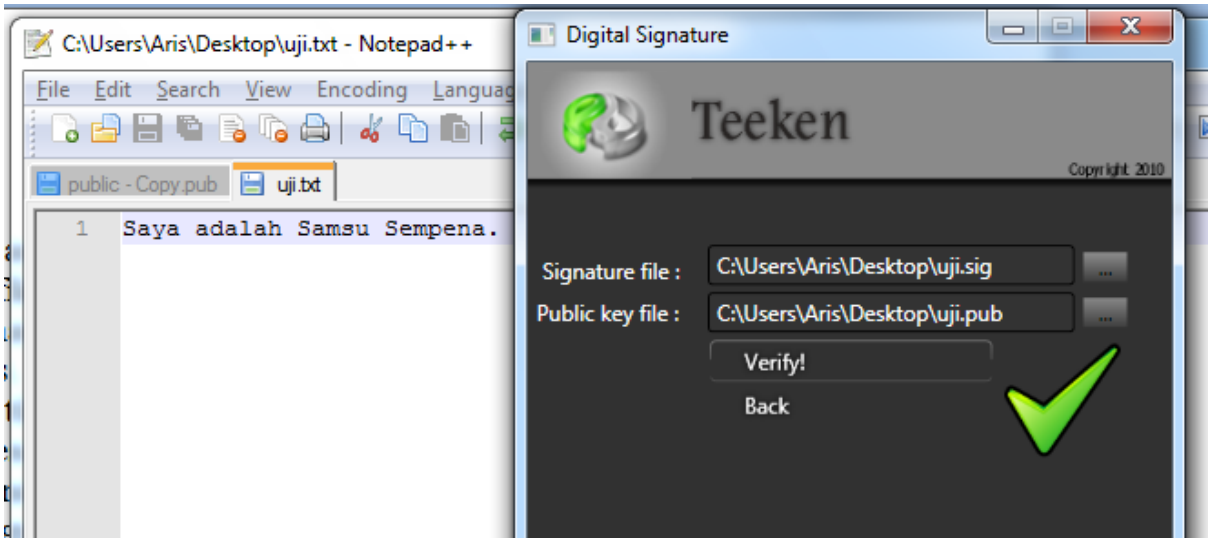
     Mengembalikan digital signature dari input yang sudah dienkripsi degan RSA.

   - `public static Boolean check(String input, BigInteger publicKey, BigInteger n,String signature)`

     Mengecek apakah hasil dekripsi signature dengan menggunakan publicKey dan n sama dengan nilai hash.

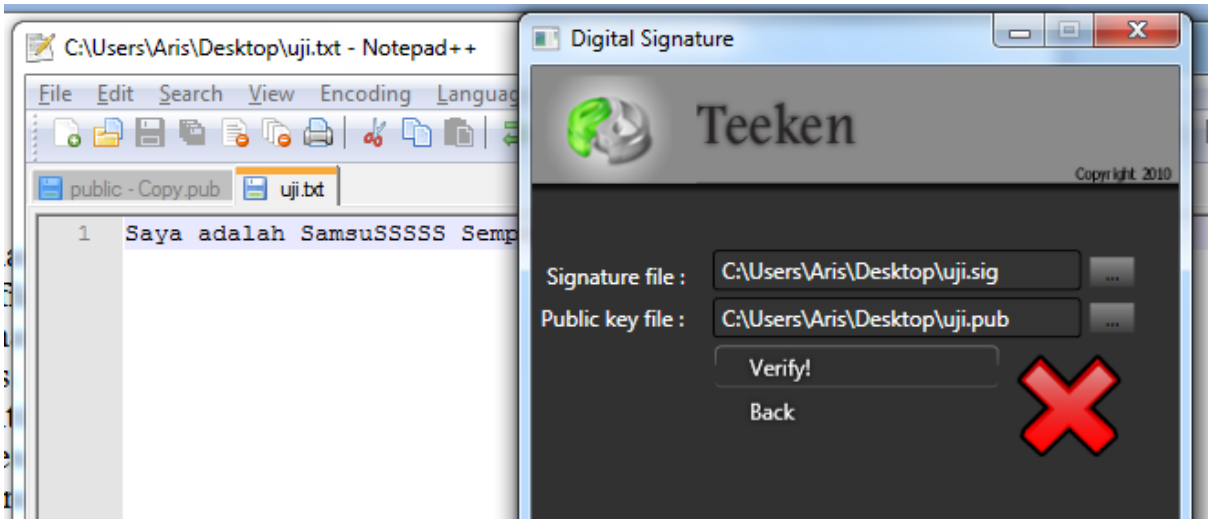
BAB 5 Pengujian dan Analisis Hasil

Pesan berhasil didekripsi dengan kunci dekripsi yang benar:



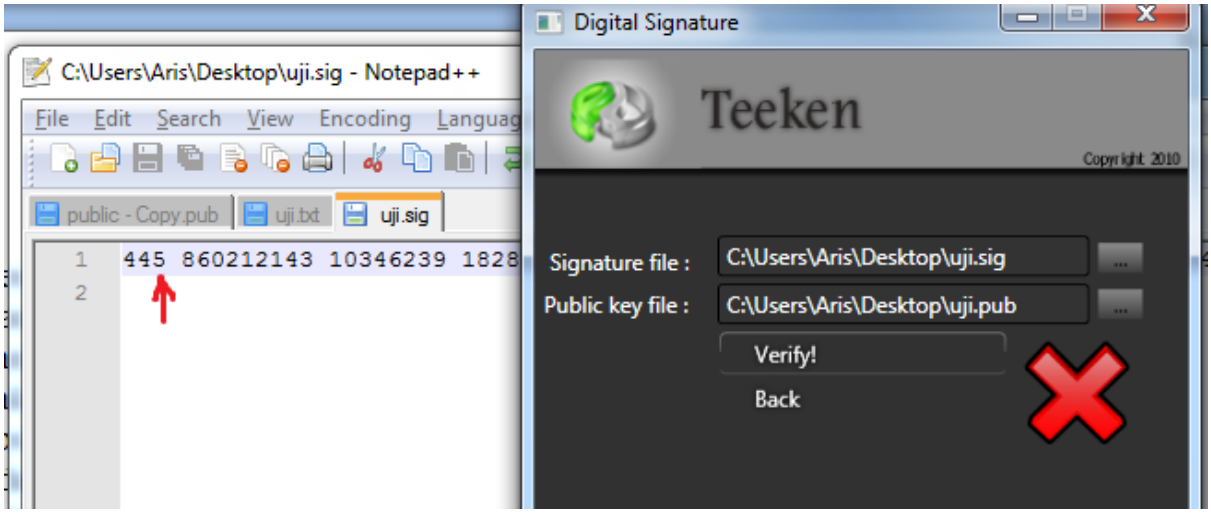
Gambar 4 Pesan Valid

Pesan ditambahkan karakter:



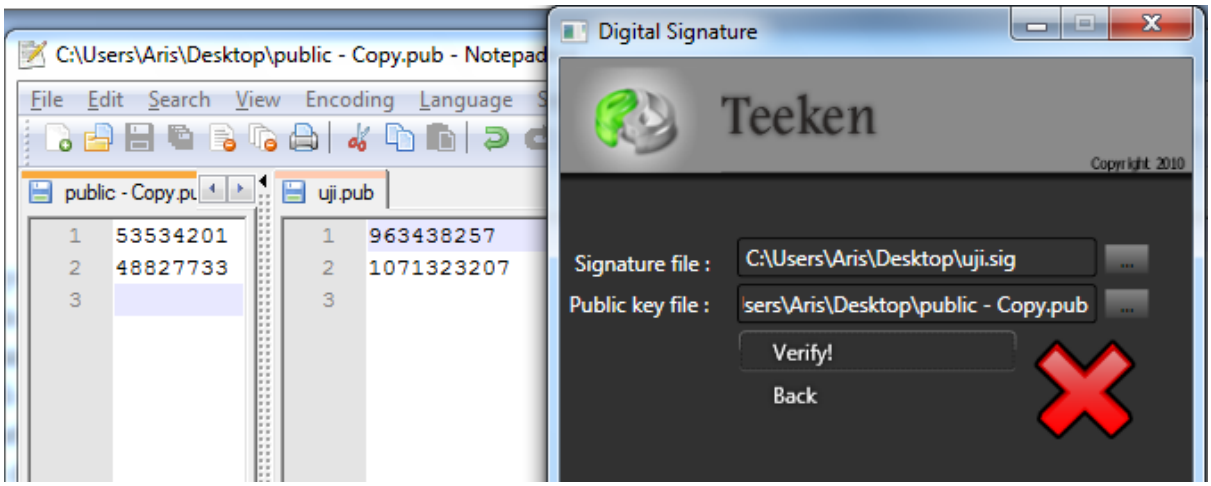
Gambar 5 Pesan Tidak Valid Karena Ditambahkan Karakter

Karakter dalam tanda tangan digital diubah:

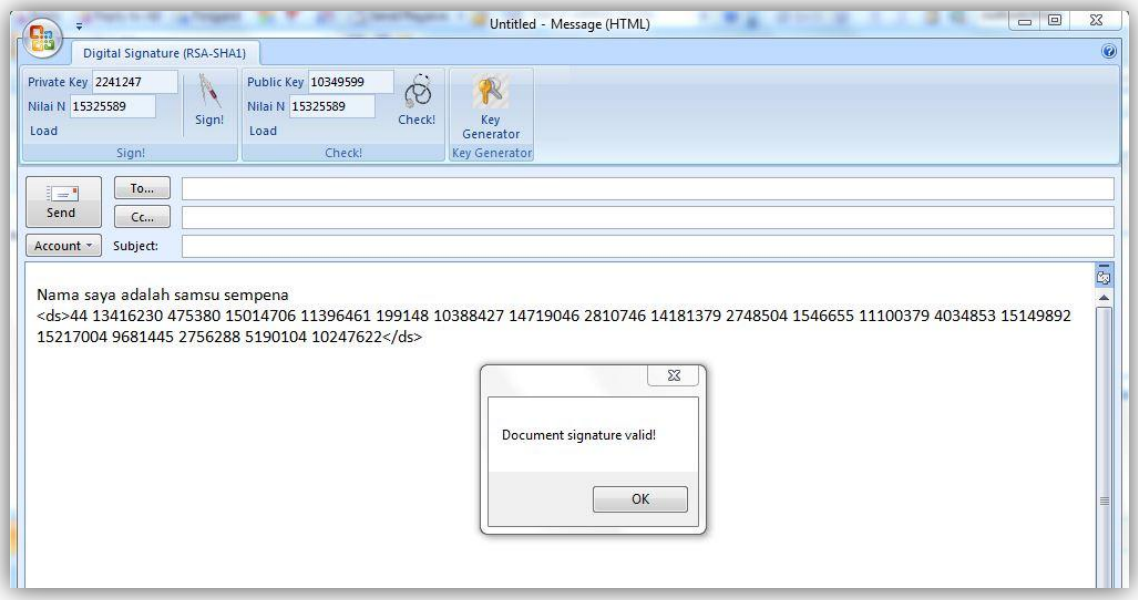


Gambar 6 Pesan Tidak Valid Karena Karakter di dalam Tanda Tangan Diubah

Kunci privat yang digunakan tidak berpadanan dengan pasangan kunci publiknya:

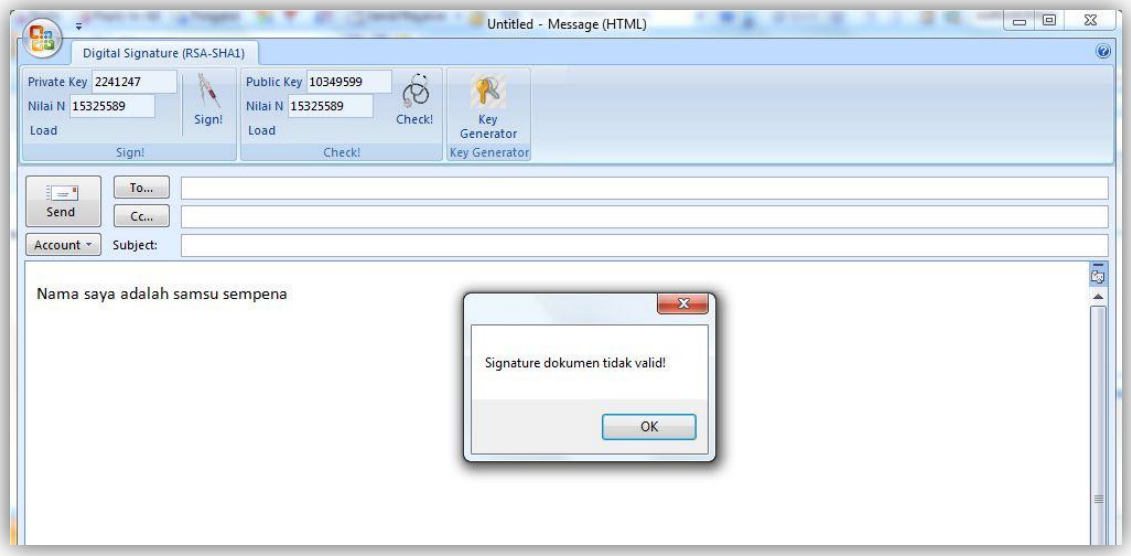


Gambar 7 Pesan Tidak Valid Karena Kunci Privat Tidak Berpadanan dengan Kunci Publik



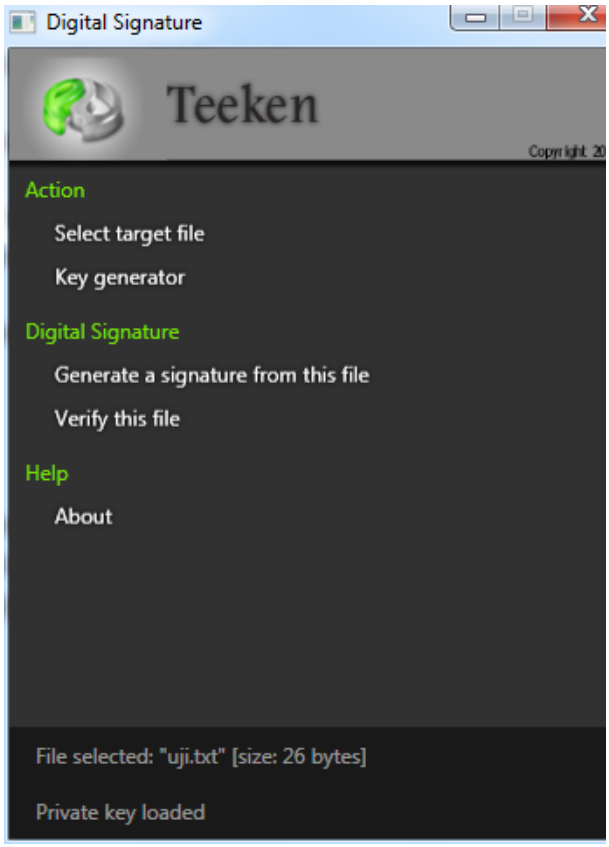
Gambar 8 Pesan Valid

Tanda tangan digital dihapus dari dokumen:



Gambar 9 Pesan Tidak Valid Karena Tanda Tangan Digital Dihapus

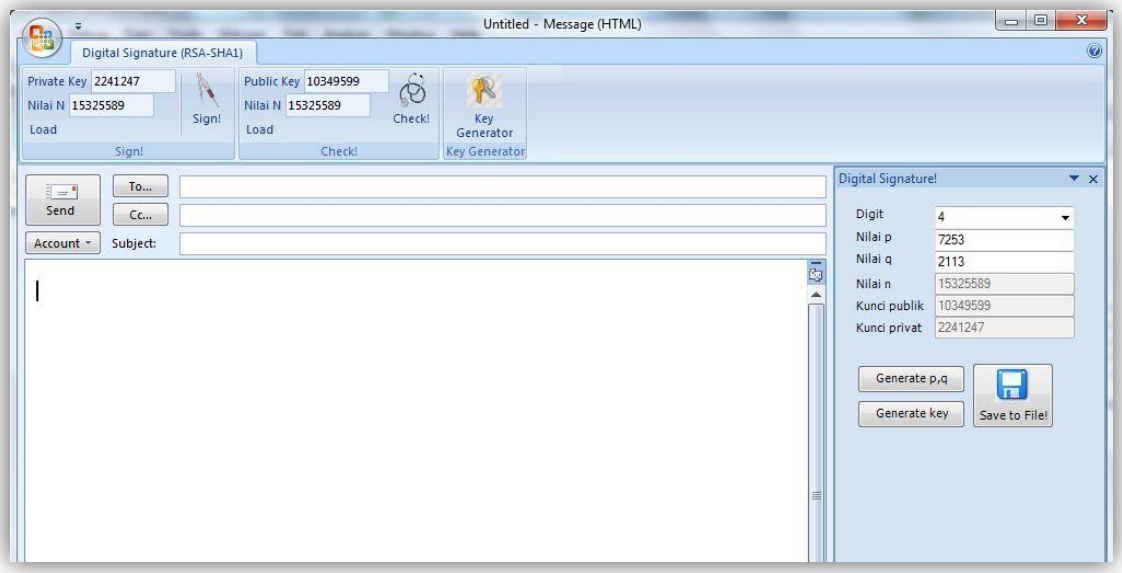
BAB 6 Lampiran



Gambar 10 Antarmuka Aplikasi Desktop



Gambar 11 Antarmuka Key Generator Aplikasi Desktop



Gambar 12. Antarmuka Add-in Pada Microsoft Outlook

File uji.txt, yaitu pesan yang akan ditandatangan

```
Saya adalah Samsu Sempena.
```

File uji.pri, yaitu kunci privat yang digunakan untuk enkripsi

```
219234473
1071323207
```

File uji.pub, yaitu kunci publik yang digunakan untuk dekripsi

```
963438257
1071323207
```

File uji.sig, yaitu file yang berisi tandatangan digital dari pesan

```
44 860212143 10346239 182810535 744981594 401012151 35257121 77497819 734571990
14524148 751086385 800643222 851157272 79600641 482131566 732897188
```

BAB 7 Kesimpulan dan Saran

7.1 Kesimpulan

1. Keamanan dan keabsahan dari suatu dokumen elektronis sangat penting, terutama jika menyangkut data yang sensitif.
2. Diperlukan suatu metode untuk memastikan keaslian data tersebut, yaitu dengan *Digital Signature* (tanda tangan digital).
3. Digital signature ini dicapai dengan menambahkan hasil hash dari data tersebut dan dienkripsi dengan kunci privat.
4. Dengan menggunakan algoritma RSA, tandatangan yang telah dienkripsi tersebut dapat didekripsi dengan kunci publik.

7.2 Saran

1. Untuk kedepannya diharapkan enkripsi/dekripsi message digest dapat digunakan algoritma selain RSA.
2. Untuk kedepannya diharapkan fungsi hash yang diimplementasikan selain SHA-1, misalnya menggunakan MD5.
3. Untuk kedepannya program add-in bisa untuk aplikasi lainnya, misalnya untuk pengiriman file melalui Yahoo! Messenger.